서브쿼리 실행시
주의할 문장

각 부서에서 급여가 가장 높은 사원의 이름(ename), 급여(sal), 부서이름(dname)을 조회하시오
(조건: **조인과 서브쿼리를 함께 사용할 것**)

❌ 오답 쿼리

```
SELECT e.ename, e.sal, d.dname
FROM Emp e
JOIN Dept d ON e.deptno = d.deptno
WHERE e.sal IN (
    SELECT MAX(sal)
    FROM Emp
    GROUP BY deptno
);

```

❌ 오답 이유

- 서브쿼리는 **부서별 최고 급여 목록**만 반환
- `e.sal IN (...)` 조건에는 **부서 조건이 없음**
- 다른 부서의 최고 급여와 **우연히 같은 급여**를 가진 사원도 포함될 수 있음
- 즉, **"각 부서에서"를 보장하지 못함**

✅ 정답 쿼리 (상관 서브쿼리)

```
SELECT e.ename, e.sal, d.dname
FROM Emp e
JOIN Dept d ON e.deptno = d.deptno
WHERE e.sal = (
    SELECT MAX(e2.sal)
    FROM Emp e2
    WHERE e2.deptno = e.deptno
);
``
```

✅ 정답 이유

- 서브쿼리에서 **같은 부서(deptno)** 기준으로 MAX(sal) 계산
- 각 부서마다 정확히 **최고 급여 사원만 선택**
- 문제 조건 완벽 충족

✅ 왜 GROUP BY만으로는 안 되는가?

문제 다시 정리

> **각 부서에서 급여가 가장 높은 사원의 이름(ename), 급여(sal), 부서이름(dname)** 을 조회하시오.

핵심은 이거야:

> ❗ **부서별 "최고 급여 값"이 아니라**
> 👉 **"그 최고 급여를 받는 사원"을 찾아야 한다**

1️⃣ 학생들이 가장 많이 하는 시도 (❌ 오답)

```sql
SELECT deptno, MAX(sal)
FROM Emp
GROUP BY deptno;
```

❌ 왜 오답인가?

- 이 쿼리의 결과는:
  - ✅ 부서별 최고 급여 **값**
  - ❌ **사원의 이름(ename)** 없음
  - ❌ 부서이름(dname) 없음

📌 문제에서 요구한 좌측 결과:

```
ename | sal | dname
```

📌 실제 결과:

```
deptno | max(sal)
```

1️⃣ 학생들이 가장 많이 하는 시도 (❌ 오답)

```sql
SELECT deptno, MAX(sal)
FROM Emp
GROUP BY deptno;
```

❌ 왜 오답인가?

- 이 쿼리의 결과는:
  - ✅ 부서별 최고 급여 **값**
  - ❌ **사원의 이름(ename)** 없음
  - ❌ 부서이름(dname) 없음

📌 문제에서 요구한 좌측 결과:

```
ename | sal | dname
```

📌 실제 결과:

```
deptno | max(sal)
```

2️⃣ 그럼 이렇게 하면 되지 않을까? (❌ 전형적 함정)

```sql
SELECT ename, sal, deptno
FROM Emp
GROUP BY deptno;
```

❌ 왜 안 되나?

- GROUP BY deptno
- 근데 ename, sal은:
  - 집계 함수도 아니고
  - GROUP BY에도 없음

👉 **SQL 문법 규칙 위반**

📌 시험용 공식:

> **SELECT에 나온 컬럼은**
> **① GROUP BY에 있거나**
> **② 집계 함수(SUM, MAX 등)에 포함돼야 한다**

3️⃣ 그럼 MAX(sal)을 같이 쓰면? (그래도 ❌)

```
SELECT ename, sal, MAX(sal)
FROM Emp
GROUP BY deptno;
``
```

❌ 왜 틀렸나?

- 여전히 ename, sal은:
  - 어느 사원의 값인지 **알 수 없음**
- DB 입장:

> "이 부서의 최고 급여는 알겠는데
> 그걸 누가 받았는지는 말 안 했잖아?"

👉 **집계 결과는 '행'을 잃어버린 상태**

4️⃣ 핵심 개념: GROUP BY는 "행 추적"이 불가능

GROUP BY가 하는 일

- 여러 행 → **1행으로 압축**
- 결과:
  - ✅ 최대값, 평균값, 합계
  - ❌ 그 값을 만든 "개별 사원 정보"

즉

> ✅ GROUP BY = **요약** ❌ GROUP BY = **사원 특정 불가**

5️⃣ 그래서 필요한 게 바로 ✅ 서브쿼리

✅ 올바른 사고 흐름

1. 각 부서의 최고 급여를 구한다
2. **그 급여를 받은 사원**을 다시 찾는다

이걸 분리해야 함 → **서브쿼리**

✅ 정답 패턴 (왜 이건 되는가)

```sql
SELECT e.ename, e.sal, d.dname
FROM Emp e
JOIN Dept d ON e.deptno = d.deptno
WHERE e.sal = (
    SELECT MAX(e2.sal)
    FROM Emp e2
    WHERE e2.deptno = e.deptno
);
```

✅ 이 쿼리가 되는 이유

- 서브쿼리:
  - 부서별 최고 급여 계산 ✅
- 바깥 쿼리:
  - 그 급여를 가진 **사원 행을 다시 찾아옴** ✅
- 결과:
  - ename, sal, dname 전부 확보 ✅

6️⃣ 시험에서 쓰는 한 문장 설명 (서술형 대비)

> **"GROUP BY는 집계 결과만 반환하며,**
> **집계값을 받은 개별 사원의 정보를 알 수 없기 때문에**
> **서브쿼리를 사용해 사원을 다시 추적해야 한다."**

2️⃣ 논리적 실행 순서 (시험에서 설명하는 방식 ✅)

🔹 ① 바깥 쿼리의 한 행(e)을 기준으로 한다

예:

```
e = (ename=SCOTT, sal=3000, deptno=20)
```

🔹 ② 서브쿼리 실행 (상관 서브쿼리!)

```
SELECT MAX(e2.sal)
FROM Emp e2
WHERE e2.deptno = e.deptno;
``
```

➡ 이때 `e.deptno = 20`

결과:

```
부서 20의 최대 급여 = 3000
```

🔹 ③ WHERE 조건 비교

```
e.sal = (부서 20의 최대 급여)
3000 = 3000 ✅
``
```

➡ 조건 만족 → 이 행을 결과 후보로 통과

🔹 ④ 그 다음 JOIN으로 부서 이름(dname) 붙임

```
JOIN Dept d ON e.deptno = d.deptno
```

➡ 결과:

```
ename | sal  | dname
SCOTT | 3000 | RESEARCH
```

🔁 이 과정을 모든 사원(e)에 대해 반복

✅ 그래서 각 부서에서 최고 급여 사원만 남음

✅ 시험용 한 문장 요약 (⭐ 매우 중요)

> 상관 서브쿼리는 바깥 쿼리의 각 행마다 실행되어
> 같은 부서의 최대 급여와 비교된다.

5️⃣ 한 줄로 최종 정리 🔥

> ✅ 조인을 먼저 하느냐, 서브쿼리를 먼저 하느냐가 아니라
> ✅ "각 사원 행마다 해당 부서의 최대 급여를 구해 비교한다"가 핵심이다

실수하기 좋은 답안 문장

4/2일 실습예제 문제 19번

(12)각 부서에서 급여가 가장 높은 사원의 이름(ename), 급여(sal), 부서이름(dname)을 조회하시오. 조인과 서브쿼리를 함께 사용할 것.
SELECT e.ename, e.sal, d.dname FROM Emp e JOIN Dept d ON e.deptno = d.deptno WHERE e.sal IN (SELECT MAX(sal) FROM Emp GROUP BY deptno) ORDER BY e.sal DESC;

✅ 문제 핵심 정리

- 각 **부서별**
- **급여가 가장 높은 사원**
- 출력: ename, sal, dname
- **조인 + 서브쿼리** 반드시 사용

✅ 가장 정석적인 정답 예시 (서브쿼리 + 조인)

✔ 상관 서브쿼리 사용 (시험에서 가장 선호됨)

```sql
SELECT e.ename, e.sal, d.dname
FROM Emp e
JOIN Dept d
  ON e.deptno = d.deptno
WHERE e.sal = (
    SELECT MAX(e2.sal)
    FROM Emp e2
    WHERE e2.deptno = e.deptno
)
ORDER BY d.dname;

```

✅ 왜 이게 정답인가?

- 서브쿼리에서 **같은 부서(deptno)** 기준으로 MAX(sal) 계산
- 바깥 쿼리의 사원과 **부서별 최고 급여 사원만 매칭**
- "각 부서에서" 조건을 정확히 충족

✅ 또 다른 정답 예시 (인라인 뷰 사용)

```sql
SELECT e.ename, e.sal, d.dname
FROM Emp e
JOIN (
    SELECT deptno, MAX(sal) AS max_sal
    FROM Emp
    GROUP BY deptno
) m
  ON e.deptno = m.deptno
 AND e.sal = m.max_sal
JOIN Dept d
  ON e.deptno = d.deptno
ORDER BY d.dname;
```

✅ 특징

- 서브쿼리에서 **부서별 최고 급여 테이블 생성**
- 조인을 통해 정확한 사원 추출
- 실무 + 시험 둘 다 잘 쓰이는 형태

❌ 네가 제시한 SQL이 위험한 이유

```sql
WHERE e.sal IN (SELECT MAX(sal) FROM Emp GROUP BY deptno)
```

문제점 🚨

- **부서 정보 없이** 최고 급여 목록만 비교
- 같은 급여가 다른 부서에 있으면 **잘못된 매칭 발생**
- 논리적으로는 "각 부서" 보장이 안 됨

✅ 그래서 시험에서는 **오답 처리되는 경우가 많음**

🎯 시험용 한 줄 결론

> **"각 부서에서"가 나오면**
> **MAX(sal) 서브쿼리에 반드시 `deptno = e.deptno` 조건이 들어가야 한다.**

오답처리되기 쉬운 유사 문장

③'대한 극장의 전체 수입을 보이시오

```
SELECT 극장.극장이름, SUM(상영관.가격) AS 전체수입 FROM 예약
    JOIN 상영관 ON 예약.극장번호 = 상영관.극장번호
           AND 예약.상영관번호 = 상영관.상영관번호
    JOIN 극장   ON 상영관.극장번호 = 극장.극장번호
    WHERE 극장.극장이름 = '대한'
    GROUP BY 극장.극장이름;
```

1. JOIN 조건을 WHERE 절에 잘못 분리한 경우 ❌

❌ 오답 예시

```sql
SELECT 극장.극장명, SUM(상영관.가격)
FROM 예약, 상영관, 극장
WHERE 예약.상영관번호 = 상영관.상영관번호
  AND 상영관.극장번호 = 극장.극장번호
  AND 극장.극장명 = '대한';
```

왜 틀릴 수 있나?

- **구문상으로는 실행되지만,**
  시험에서는 **명시적 JOIN을 요구**하는 문제가 많음
- 특히 "JOIN을 사용하라"는 조건이 있는 경우 **오답 처리**

2. 극장 조건을 JOIN 절에 넣지 않고 WHERE에만 둔 경우

⚠️ 부분 오답 예시

```sql
SELECT 극장.극장이름, SUM(상영관.가격)
FROM 예약
JOIN 상영관 ON 예약.상영관번호 = 상영관.상영관번호
JOIN 극장 ON 상영관.극장번호 = 극장.극장번호
GROUP BY 극장.극장이름
HAVING 극장.극장이름 = '대한';
```

문제점

- 결과는 나오지만:
  - **모든 극장의 수입을 먼저 계산한 뒤**
  - '대한'만 필터링
- 문제에서
  👉 **"대한 극장의 전체 수입을 보이시오"** 라면
  👉 **WHERE 절에서 먼저 제한**하는 것이 정석

✅ DB 이론/시험에서는 **연산 순서**를 중요하게 봄

3. GROUP BY 컬럼 누락 ❌ (자주 나오는 치명적 실수)

❌ 오답 예시

```sql
SELECT 극장.극장이름, SUM(상영관.가격)
FROM 예약
JOIN 상영관 ON 예약.상영관번호 = 상영관.상영관번호
JOIN 극장 ON 상영관.극장번호 = 극장.극장번호
WHERE 극장.극장이름 = '대한';
```

왜 틀렸나?

- SUM() 같은 **집계 함수** 사용 시
- **집계되지 않은 컬럼은 반드시 GROUP BY에 포함**

✅ DBMS에 따라 실행은 되기도 하지만

✅ **이론 문제 / 시험에서는 100% 오답 처리**

4. SUM 대상 컬럼을 잘못 선택한 경우 ❌

❌ 오답 예시

```sql
SUM(예약.좌석번호)
```

또는

```sql
SUM(상영관.상영관번호)
```

문제점

- 문제는 "**수입**"
- 수입 = **가격(요금)** 의 합
- 식별자(PK/FK)를 SUM → **의미적으로 완전 오답**

✅ 숫자 컬럼이라도 **의미가 맞지 않으면 틀림**

5. DISTINCT를 잘못 사용한 경우 ⚠️

⚠️ 오답 예시

```
SELECT 극장.극장이름, SUM(DISTINCT 상영관.가격)
```

왜 위험한가?

- 같은 가격의 상영관/예약이 여러 번 있으면
- **중복 가격이 제거되어 실제 수입보다 작아짐**

✅ 수입 계산에서는 **DISTINCT 거의 사용하지 않음**

6. 테이블 연결 경로를 잘못 설정한 경우 ❌

❌ 오답 예시

```
JOIN 극장 ON 예약.극장번호 = 극장.극장번호
```

문제점

- 예약 테이블에 **극장번호가 없는 구조**일 가능성 큼
- 정석 경로:

```
예약 → 상영관 → 극장
```

✅ **ERD 기준 관계 무시**는 시험에서 바로 오답